



Tema 6. Bases de datos NO relacionales

Índice

1.- Introducción a NOSql.....	2
1.1.- MongoDB.....	3
1.2.- MongoDB Compass.....	4
2.- Documentos y tipos de datos.....	5
2.1.- Sintaxis JSON.....	5
2.2.- Tipos de datos JSON.....	6
2.3.- Documentos de MongoDB.....	7
2.4.- Tipos de datos de MongoDB.....	7
3.- Bases de datos en MongoDB.....	10
3.1.- Consulta de documentos.....	10
3.2.- Insertar documentos.....	17
3.3.- Eliminar documentos.....	17
3.4.- Reemplazar y actualizar documentos.....	18
3.5.- Operaciones con arrays.....	20
4.- Data Aggregation.....	21
5.- Joins \$lookup.....	22

En esta unidad, estudiaremos otro enfoque de las bases de datos, las NO relacionales.

1.- Introducción a NOSql

Hay dos tipos de bases de datos: bases de datos relacionales y no relacionales.

Las bases de datos **no relacionales** a menudo se denominan bases de datos **NoSQL**.

Una base de datos NoSQL se utiliza para almacenar grandes cantidades de datos complejos y diversos, como catálogos de productos, registros, interacciones de usuarios, análisis y más.





1.1.- MongoDB

MongoDB es una popular base de datos **NoSQL** que puede almacenar datos estructurados y no estructurados. Fundada en 2007 por Kevin P. Ryan, Dwight Merriman y Eliot Horowitz en Nueva York, la organización se llamó inicialmente 10gen y luego se renombró como MongoDB, una palabra inspirada en el término **humongous**.

Su diseño basado en **documentos** y su sintaxis intuitiva para consultas y comandos hace que sea fácil de aprender.

Descarga e instalación de MongoDB:

- **Paso 1: Descargar e instalar la versión Community Server**

Ir al página oficial de MongoDB <https://www.mongodb.com/try/download/community>

- **Paso 2. Encender el servidor MongoDB**

Una opción fácil para encender el servidor es ir a la carpeta del ordenador donde se instaló Mongo y abrir el archivo **mongod**.

Dentro de la carpeta **bin** se encuentra los archivos ejecutables: **mongod** y **mongo**

C:\Program Files\MongoDB\Server\4.4\bin

Antes de abrir el archivo **mongod** hay que hacer un paso previo: Crear en el **disco C** una carpeta llamada **data** y dentro de esta otra carpeta llamada **db**.

C:\data\db

Ahora si, al abrir el archivo **mongod** y saldrá la clásica pantalla negra.

```
C:\Program Files\MongoDB\Server\4.4\bin>mongod.exe
[RECOVERY_PROGRESS] Set global oldest timestamp: (0, 0)
{"t":{"$date":"2021-06-04T12:15:39.464-05:00"},"s":"I", "c":"STORAGE", "id":22430, "ctx":"initandlisten","msg":"Wire
dTiger message","attr":{"message":"[1622826939:464382][10588:140735344694608], WT_SESSION.checkpoint: [WT_VERB_CHECKPOIN
T_PROGRESS] saving checkpoint snapshot min: 1, snapshot max: 1 snapshot count: 0, oldest timestamp: (0, 0) , meta checkp
oint timestamp: (0, 0)"}
{"t":{"$date":"2021-06-04T12:15:39.602-05:00"},"s":"I", "c":"STORAGE", "id":4795906, "ctx":"initandlisten","msg":"Wire
dTiger opened","attr":{"durationMillis":915}}
{"t":{"$date":"2021-06-04T12:15:39.602-05:00"},"s":"I", "c":"RECOVERY", "id":23987, "ctx":"initandlisten","msg":"Wire
dTiger recoveryTimestamp","attr":{"recoveryTimestamp":{"$timestamp":{"t":0,"i":0}}}}
{"t":{"$date":"2021-06-04T12:15:39.608-05:00"},"s":"I", "c":"STORAGE", "id":4366408, "ctx":"initandlisten","msg":"No t
able logging settings modifications are required for existing WiredTiger tables","attr":{"loggingEnabled":true}}
{"t":{"$date":"2021-06-04T12:15:39.612-05:00"},"s":"I", "c":"STORAGE", "id":22262, "ctx":"initandlisten","msg":"Time
stamp monitor starting"}
{"t":{"$date":"2021-06-04T12:15:39.693-05:00"},"s":"W", "c":"CONTROL", "id":22120, "ctx":"initandlisten","msg":"Acce
ss control is not enabled for the database. Read and write access to data and configuration is unrestricted","tags":["st
artupWarnings"]}
{"t":{"$date":"2021-06-04T12:15:39.694-05:00"},"s":"W", "c":"CONTROL", "id":22140, "ctx":"initandlisten","msg":"This
server is bound to localhost. Remote systems will be unable to connect to this server. Start the server with --bind_ip
<address> to specify which IP addresses it should serve responses from, or with --bind_ip_all to bind to all interfaces.
If this behavior is desired, start the server with --bind_ip 127.0.0.1 to disable this warning","tags":["startupWarning
s"]}
{"t":{"$date":"2021-06-04T12:15:39.725-05:00"},"s":"I", "c":"STORAGE", "id":20536, "ctx":"initandlisten","msg":"Flow
Control is enabled on this deployment"}
{"t":{"$date":"2021-06-04T12:15:40.555-05:00"},"s":"I", "c":"FTDC", "id":20625, "ctx":"initandlisten","msg":"Init
ializing full-time diagnostic data capture","attr":{"dataDirectory":"C:/data/db/diagnostic.data"}}
{"t":{"$date":"2021-06-04T12:15:40.559-05:00"},"s":"I", "c":"NETWORK", "id":23015, "ctx":"listener","msg":"Listening
on","attr":{"address":"127.0.0.1"}}
{"t":{"$date":"2021-06-04T12:15:40.559-05:00"},"s":"I", "c":"NETWORK", "id":23016, "ctx":"listener","msg":"Waiting f
or connections","attr":{"port":27017,"ssl":"off"}}
```



En esta pantalla mostrará que el servidor está encendido y está esperando que alguien se conecte. (No debes cerrar esta pantalla)

- **Paso 3. Abrir la consola y conectarse al servidor**

Ahora se deberá abrir el otro ejecutable: **mongo**. Este ejecutable es una **consola o terminal**.

Escribimos en la consola **2+2** para probar que funciona.

```
C:\Program Files\MongoDB\Server\4.4\bin>mongo.exe
MongoDB shell version v4.4.6
connecting to: mongodb://127.0.0.1:27017/?compressors=disabled&gssapiServiceName=mongodb
Implicit session: session { "id" : UUID("7cb86bd3-afcb-4a9f-9ee5-a0fb2a4416a6") }
MongoDB server version: 4.4.6
---
The server generated these startup warnings when booting:
  2021-06-04T11:47:21.131-05:00: Access control is not enabled for the database. Read and write access to data and
configuration is unrestricted
  2021-06-04T11:47:21.132-05:00: This server is bound to localhost. Remote systems will be unable to connect to th
is server. Start the server with --bind_ip <address> to specify which IP addresses it should serve responses from, or wi
th --bind_ip_all to bind to all interfaces. If this behavior is desired, start the server with --bind_ip 127.0.0.1 to di
sable this warning
---
---
  Enable MongoDB's free cloud-based monitoring service, which will then receive and display
metrics about your deployment (disk utilization, CPU, operation statistics, etc).

  The monitoring data will be available on a MongoDB website with a unique URL accessible to you
and anyone you share the URL with. MongoDB may use this information to make product
improvements and to suggest MongoDB products and deployment options to you.

  To enable free monitoring, run the following command: db.enableFreeMonitoring()
  To permanently disable this reminder, run the following command: db.disableFreeMonitoring()
---
> 2+2
4
>
```

Además de la consola clásica hay aplicaciones de entorno visual para comodidad del usuario como **MongoDB Compass**.

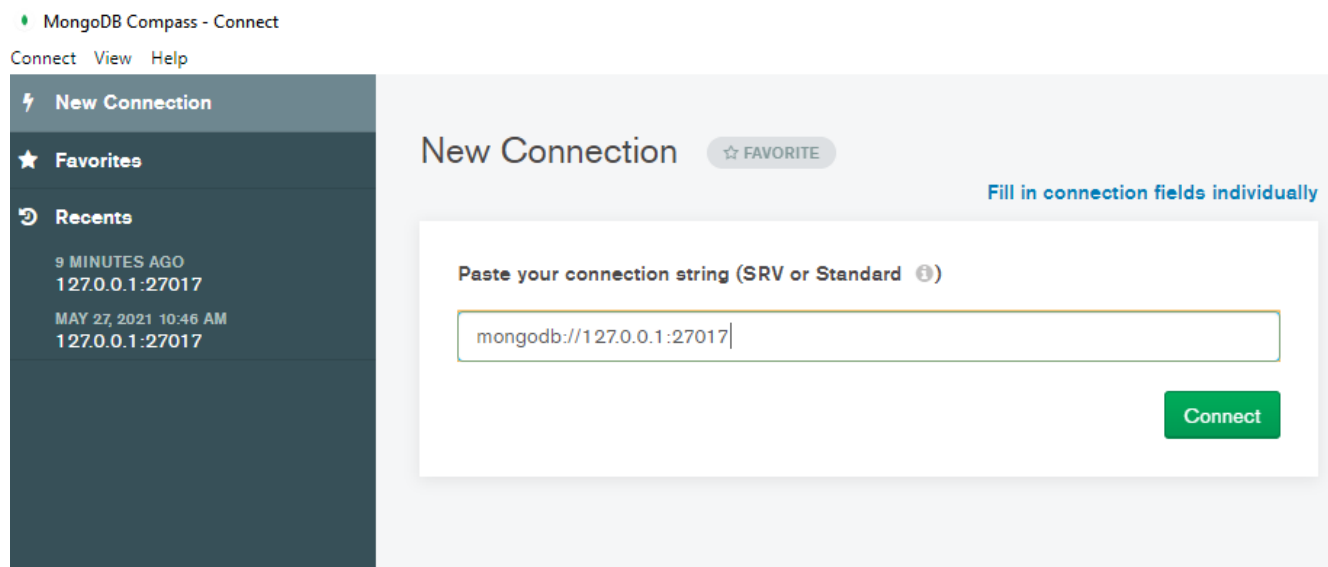
1.2.- MongoDB Compass

MongoDB Compass es un entorno visual e intuitivo para interactuar con el servidor. Generalmente se instala junto cuando se instala MongoDB.

Para usar MongoDB Compass se debe abrir el programa y conectarse al servidor de mongo. Para esto hay que copiar el string que apareció previamente en la consola:

```
mongodb://127.0.0.1:27017
```

Luego clic en Connect y listo. Ahora podrás crear su propia base de datos o subir un archivo de base de datos.



En la sección 3 se verá cómo crear una base de datos mediante la consola y con la interfaz MongoDB Compass.

2.- Documentos y tipos de datos

Una de las características de **MongoDB** es su modelo de datos basado en **documentos**, que son aceptados como una forma flexible de transportar información.

Muchas aplicaciones que intercambian datos en forma de documentos **JavaScript Object Notation (JSON)**. MongoDB almacena datos en formato **JSON binario (BSON)** y los representa en JSON legible por humanos.

2.1.- Sintaxis JSON

Los documentos u objetos **JSON** son un conjunto de texto sin formato de pares **clave-valor**.

JSON tiene una estructura de un conjunto de llaves { }, corchetes [], dos puntos : y comas , .

Clave : Valor

En un objeto JSON, los pares **clave-valor** están encerrados con llaves { }. La **clave** es siempre una cadena de texto y el **valor** puede ser cualquier **tipo** especificado por JSON.

```
{ key : value }
```

Array

Un **array** es un conjunto de valores encerrados entre corchetes [] y separados por comas , .

```
[ value1, value2, value3 ]
```



Ejemplo de un documento JSON que contiene la información básica de una empresa:

```
{
  "company_name" : "Sparter",
  "founded_year" : 2007,
  "twitter_username" : null,
  "address" : "15 East Street",
  "no_of_employees" : 7890,
  "revenue" : 879423000
}
```

2.2.- Tipos de datos JSON

JSON es un formato de intercambio de datos. La presencia de tipos de datos básicos proporcionados por JSON reduce la complejidad durante este proceso. Por eso se mantiene simple y mínimo en términos de tipos de datos.

- String
- Number
- Boolean
- Object
- Array
- Null

Un número es solo una secuencia de dígitos. No distingue entre números como enteros o flotantes.

JSON no admite un tipo de datos Fecha y estas solo se representan como cadenas simples. Ejemplos:

```
{
  "title": "A Swedish Love Story",
  "released": "1970-04-24"
}
```

Las partes que intercambian la información deben estandarizar el formato fecha durante las transferencias. La forma de leer los datos depende de los intérpretes de los lenguajes y de sus contratos de intercambio de datos.



2.3.- Documentos de MongoDB

Una base de datos MongoDB se compone de **colecciones** y **documentos**. Una base de datos puede tener una o más colecciones, y cada colección puede almacenar uno o más documentos relacionados.

En comparación con RDBMS, las **colecciones** son análogas a las **tablas** y los **documentos** son análogos a las **filas** dentro de una tabla. Sin embargo, los documentos son mucho más flexibles en comparación con las filas de una tabla.

2.4.- Tipos de datos de MongoDB

String: en MongoDB, los campos de cadena están codificados en UTF-8. Además, admiten capacidades de búsqueda con expresiones regulares.

Numbers: En MongoDB se admite los varios tipos de números.

- **double:** punto flotante de 64 bits
- **int:** entero de 32 bits con signo
- **long:** entero sin signo de 64 bits
- **decimal:** punto flotante de 128 bits, que cumple con IEE 754

Booleans: se utiliza para representar si algo es verdadero o falso.

Null: Nulo es un tipo de datos especial en un documento y denota un campo que no contiene un valor. El campo nulo solo puede tener nulo como valor.

Objects: Se utilizan para representar documentos **ANIDADOS** o **INCRUSTADOS**, es decir, un campo cuyo valor es otro documento JSON válido.

El siguiente documento tiene otro documento anidado llamado **"host"**

El valor del campo de host es otro documento de JSON. MongoDB usa una notación de puntos . para acceder a los objetos anidados.

Para acceder a un documento anidado, podemos crear una variable y a partir de ahí usar la notación de punto para acceder al documento anidado.



```
{
  "listing_url": "https://www.airbnb.com/rooms/1001265",
  "name": "Ocean View Waikiki Marina w/prkg",
  "summary": "A great location that work perfectly for business.",
  "host": {
    "host_id": "5448114",
    "host_name": "David",
    "host_location": "Honolulu, Hawaii, United States"
  }
}
```

Ejemplo: Acceder la información del **host_name** del documento **listing**.

"**host_name**" es un campo que está dentro del documento "**host**" que a su vez es un documento anidado del documento principal **listing** almacenado en una variable.

```
var listing = {
  "listing_url": "https://www.airbnb.com/rooms/1001265",
  "name": "Ocean View Waikiki Marina w/prkg",
  "summary": "A great location that work perfectly for business.",
  "host": {
    "host_id": "5448114",
    "host_name": "David",
    "host_location": "Honolulu, Hawaii, United States"
  }
}
```

`listing.host.host_name`

Array: es una colección de cero o más valores que deben estar encerrado con corchetes []. En MongoDB, no hay límite para la cantidad de elementos que puede contener un array o la cantidad de arrays que puede tener un documento. Sin embargo, el tamaño total del documento no debe exceder los 16 MB.

```
var doc = { first_array: [ 4, 3, 2, 1 ] }
```

Se puede acceder a cada elemento de un array utilizando su posición de **índice**. El número de índice se incluye entre corchetes. (Los índices empiezan desde 0).

`doc.first_array[3]`



Un array puede contener cualquier campo de tipo de datos válido de MongoDB, incluso arrays anidados. Esto se puede ver en el siguiente ejemplo:

```
[ "this", "is", "a", "text" ] // array of strings  
  
[ 1.1, 3.2, 553.54 ] // array of doubles  
  
[ { "a" : 1 }, { "a" : 2, "b" : 3 }, { "c" : 1 } ] // array of json objects  
  
[ 12, "text", 4.35, [ 3, 2 ], { "type" : "object" } ] // array of mixed elements
```

ObjectId: Cada documento de una colección debe tener un **_id** que contenga un valor único. Este campo actúa como clave principal para estos documentos. Las claves primarias se utilizan para identificar de forma única los documentos y siempre están indexadas. El valor del campo **_id** debe ser único en una colección.

Si inserta un documento sin un campo **_id**, el controlador MongoDB generará automáticamente una ID única y la agregará al documento. Cuando el controlador agrega automáticamente el campo **_id**, el valor se genera mediante **ObjectId**.

El valor **ObjectId** está diseñado para generar código ligero que es único en diferentes máquinas. Genera un valor único de 12 bytes, donde los primeros 4 bytes representan la marca de tiempo, los bytes 5 a 9 representan un valor aleatorio y los últimos 3 bytes son un contador incremental.

```
var uniqueID = new ObjectId ()  
uniqueID
```

Fechas: MongoDB si admite tipos de fecha explícitamente. Puesto que en JSON no admiten tipos de fecha porque se representan como cadenas sin formato.

Las fechas de MongoDB se almacenan en forma de milisegundos desde el 1 de enero de 1970. Para almacenar la representación de milisegundos de una fecha, MongoDB usa un entero de 64 bits (long). Todas las fechas se almacenan en UTC y no hay una zona horaria asociada a ellas.

Puede crear instancias de fecha usando **Date ()** , **new Date ()** o **new ISODate ()**

Las fechas creadas con un **new Date ()** o **new ISODate ()** siempre estarán en UTC, y las creadas con **Date ()** estarán en la zona horaria local.

```
var date = Date() // Sat Sept 03 1989 07:28:46 GMT-0500 (CDT)  
  
var date = new Date() //ISODate("1989-09-03T10:11:23.357Z")  
var isoDate = new ISODate() //ISODate("1989-09-03T11:13:26.442Z")
```



Date() usa la representación de fecha de JavaScript, que está en forma de cadenas simples. No son útiles para la comparación o manipulación.

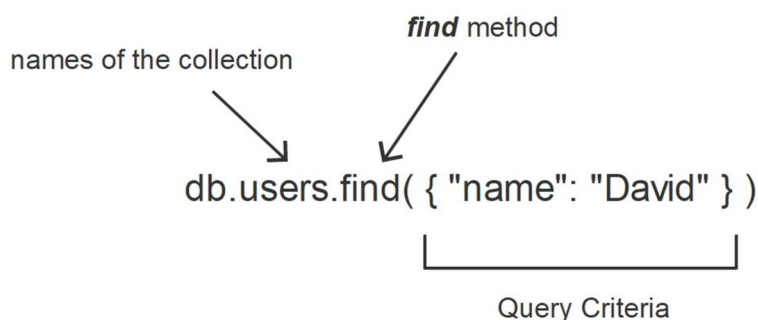
3.- Bases de datos en MongoDB

MongoDB tiene una consola de administración de línea de comandos. Algunos de los comandos más utilizados son:

- **cls** : limpia la consola.
- **show dbs** : muestra todas las bases de datos almacenadas en MongoDB que tienen alguna colección.
- **db** : muestra en que base de datos estamos trabajando.
- **use [nombreBD]** : seleccionamos la base de datos a usar. **Si no existe la CREA.**
- **show collections** : muestra las colecciones (TABLAS) de la base de datos en uso.
- **db.createCollection('coleccion1')** : crea una colección de nombre coleccion1 en la base de datos en uso.
- **db.coleccion1.drop()** : elimina la colección coleccion1 de la base de datos en uso.
- **db.dropDatabase()** : elimina la base de datos en uso.

3.1.- Consulta de documentos

Las consultas de MongoDB se basan en documentos JSON. La imagen es un ejemplo de una consulta simple de MongoDB que encuentra todos los documentos de la colección "users" donde el campo de **name** contiene el valor **David**:



Consultas básicas de MongoDB

- **find** mostrará todos los documentos de la colección.

```
db.comments.find()
```



- **find** con parámetros devolverá solo los documentos que cumplan la condición.

```
db.comments.find({"name" : "Lauren Carr"})
```

Podemos añadir `.pretty()` al final para que formatee los datos.

```
db.comments.find({"name" : "Lauren Carr"}).pretty()
```

- **findOne** muestra solo un registro coincidente, devolviendo solo el primero.

`db.comments.findOne ()`

```
db.comments.findOne()
```

- **Projection** permite incluir o excluir campos específicos del resultado. Se puede excluir explícitamente un campo configurándolo en 0 o incluir con 1.

```
db.comments.find(
  {"name" : "Lauren Carr"},
  {"name" : 1, "date": 1}
).pretty()
```

Resultado:

```
{
  "_id" : ObjectId("5a9427648b0beebeb6962d2d"),
  "name" : "Lauren Carr",
  "date" : ISODate("1978-03-25T06:29:47Z")
}
{
  "_id" : ObjectId("5a9427648b0beebeb6962d2e"),
  "name" : "Lauren Carr",
  "date" : ISODate("1986-10-26T11:31:17Z")
}
{
  "_id" : ObjectId("5a9427648b0beebeb6962d2f"),
  "name" : "Lauren Carr",
  "date" : ISODate("1972-03-23T02:37:03Z")
}
```

El campo `_id` siempre se incluirá, a menos que se excluya explícitamente.

- **distinct** encuentra todos los valores únicos de un campo.

```
db.movies.distinct("rated")
```



- **countDocuments** cuenta el número de documentos de una colección o de documentos que cumplen con una condición determinada.

```
db.movies.countDocuments({"year":1999})
```

Operadores condicionales

- **Igualdad (eq):**

```
db.movies.find( {"num_mflix_comments" : 5} )  
db.movies.find( {"num_mflix_comments": {$eq : 5}} )
```

- **Desigual (neq):**

```
db.movies.find( {"num_mflix_comments" : {$ne : 5} } )
```

- **Mayor que (gt) o mayor o igual que (gte):**

```
db.movies.find(  
  {"released" :  
    {$gte: new Date('2000-01-01')}}  
).count()
```

- **Menor que (lt) o menor o igual que (lte):**

```
db.movies.find( {"num_mflix_comments" : {$lte : 2} } ).count()
```

- **Contiene (in) y no contiene (nin):**

```
db.movies.find(  
  {"rated" :  
    {$in : ["G", "PG", "PG-13"]}  
  }  
)
```

NOTA: un campo inexistente siempre tiene un valor de nulo, por lo que la condición \$nin no se cumple para ninguno documentos.



```
db.movies.countDocuments(  
  {"nef" :  
    {$nin: ["a value", "another value", null ]}  
  }  
)
```

Operadores lógicos

- **Y (and)** puede tener cualquier número de condiciones envueltas en un array y el operador devolverá solo los documentos que satisfacen todas las condiciones.

```
db.movies.countDocuments(  
  {$and :  
    [{"rated" : "UNRATED"}, {"year" : 2008}]  
  }  
)
```

En las consultas de MongoDB, el operador **\$and** está implícito y se incluye de forma predeterminada si un documento de consulta tiene más de una condición.

```
db.movies.countDocuments (  
  {"rated": "UNRATED", "year" : 2008}  
)
```

- **O (or)** puede tener cualquier número de condiciones envueltas en un array y el operador devolverá los documentos que satisfacen al menos una de las condiciones.

```
db.movies.find(  
  {$or:[  
    {"rated" : "G"},  
    {"year" : 2005},  
    {"num_mflix_comments" : {$gte : 5}}  
  ]}  
)
```

- **No (not)** representa la operación lógica NOT que niega la condición dada.

```
db.movies.find(  
  {"num_mflix_comments" :  
    {$not: {$gte : 5}}  
  }  
)
```



Expresiones regulares

En las consultas de MongoDB, las expresiones regulares se pueden usar con el operador **\$regex**.

```
db.movies.find(  
  {"title" : {$regex : "Opera"}},  
  {"title" : 1}  
)
```

Para buscar solo las cadenas que comienzan con la expresión regular dada se utiliza el operador (^).

Para hacer coincidir las cadenas que terminan con la expresión regular dada.

La búsqueda con expresiones regulares distingue entre mayúsculas y minúsculas de forma predeterminada. Las mayúsculas y minúsculas de los caracteres en el patrón de búsqueda proporcionado coincide exactamente.

El operador **\$options** es para hacer búsquedas de expresiones regulares que no distinguen entre mayúsculas y minúsculas. El argumento \$options con un valor de i, donde i significa que no distingue entre mayúsculas y minúsculas.

```
db.movies.find(  
  {"title" :  
    {"$regex" : "the", $options: "i"}  
  }  
)
```

Consultas de Arrays y Documentos Anidados

Consultar un array es similar a consultar cualquier otro campo.

```
db.movies.find ( {"cast": "Charles Chaplin"}, {"cast": 1, "_id": 0})
```

Cuando busca un campo de array utilizando un array, los elementos y su orden deben coincidir.

```
db.movies.find(  
  {"languages" : ["English", "German"]}, {"languages" : 1}  
)
```

Cuando se buscan campos de array utilizando un array, el valor se compara mediante una verificación de igualdad. Dos array cualesquiera solo pasan la verificación de igualdad si tienen los mismos elementos en el mismo orden. Por lo tanto, las dos consultas siguientes no son iguales y devolverán resultados diferentes.



- **Todos (all)** busca todos aquellos documentos donde el valor del campo contiene todos los elementos, independientemente de su orden o tamaño.

```
db.movies.find(  
  {"languages":{  
    "$all" :[ "English", "French", "Cantonese"]  
  }},  
  {"languages" : 1, "_id": 0}  
)
```

Puede buscar una array por un valor de elemento y usar la proyección para excluir todos excepto el elemento coincidente del array usando el operador \$

```
db.movies.find(  
  {"languages" : "Syriac"},  
  {"languages.$" :1}  
)
```

- **Recortar (slice)** se utiliza para limitar los elementos del array en función de su posición de índice. Este operador se puede utilizar con cualquier campo de array, independientemente del campo que se esté consultando o no. Esto significa que puede consultar un campo diferente y todavía utilizan este operador para limitar los elementos de los campos array.

```
db.movies.find(  
  {"title" : "Youth Without Youth"},  
  {"languages" : {$slice : 3}}  
)  
.pretty()
```

Nota, si indicamos un valor negativo, devolverá los últimos elementos del array. También se pueden indicar rangos usando [x, y]

Consultas de objetos anidados

Los objetos anidados también se pueden representar como valores de un campo. Por lo tanto, los campos que tienen otros objetos como valores se pueden buscar utilizando el **objeto completo como valor**.



```
db.movies.find(
  {
    "awards": {
      "wins": 1, "nominations": 0, "text": "1 win."
    },
    "awards": 1
  }
)
```

Cuando se buscan campos de objeto anidados con valores de objeto, debe haber una coincidencia exacta. Los pares clave-valor, junto con el orden de los campos, deben coincidir exactamente.

Consultas de Arrays y Documentos Anidados

Con la función `limit` se puede restringir el tamaño del resultado. Ejemplo: Limitar el resultado a 3 registros.

```
db.movies.find(
  {
    "cast" : "Charles Chaplin",
    "title": 1, "_id" :0
  }
).limit(3)
```

Excluir documentos con Skip

La función **Skip*** se utiliza para excluir algunos documentos del conjunto de resultados y devolver el resto. El cursor MongoDB proporciona la función `skip()`, que acepta un número entero y omite el número especificado de documentos del cursor, devolviendo el resto.

```
db.movies.find(
  {
    "cast" : "Charles Chaplin",
    "title": 1, "_id" :0
  }
).skip(2)
```

Ordenando documentos con sort

La ordenación de documentos se puede realizar en varios campos y cada campo puede tener un ordenamiento diferente.

```
db.movies.find(
  {
    "cast" : "Charles Chaplin",
    "title" : 1, "_id" :0
  }
).sort({"title" : 1})
```

Para ordenar en forma **descendente** se especifica con el valor -1.



3.2.- Insertar documentos

La función **insert()** se utiliza para crear un nuevo documento en una colección. Cuando se ejecuta un comando de inserción de documento, MongoDB también creará la colección dada, si aún no existe.

```
db.new_movies.insert({"_id" : 1, "title" : "Dunkirk"})
```

Para insertar varios documentos, es preferible usar la función **insertMany()**, porque la inserción ocurre como una sola operación.

```
db.new_movies.insertMany([
  {"_id" : 2, "title": "Baby Driver"},
  {"_id" : 3, "title": "Logan"},
  {"_id" : 4, "title": "John Wick: Chapter 2"},
  {"_id" : 5, "title": "A Ghost Story"}
])
```

El valor expresado por el campo **_id** es una clave principal, por lo que debe ser único. Si intenta insertar un documento cuya clave ya está presente en la colección, obtendrá un error de clave duplicada. Esto aplica también cuando se usa **insertMany**.

Insertar sin id

MongoDB verifica la presencia y unicidad de una clave primaria dada y, si la clave primaria aún no está presente, la base de datos la genera automáticamente y la agrega al documento.

```
db.new_movies.insert({"título": "Thelma"})
db.new_movies.find({"título": "Thelma"})
```

3.3.- Eliminar documentos

La función **deleteOne** se usa para eliminar un solo documento de una colección. Como el método elimina solo un documento, el valor de respuesta **deletedCount** es 1. Si la condición de consulta dada coincide con más de un documento en la colección, **solo se eliminará el primer documento**.

```
db.new_movies.deleteOne({"_id": 1})
```

La función **deleteMany** sirve para eliminar varios documentos en un solo comando. Debe proporcionarse con una condición de consulta, y se eliminarán todos los documentos que coincidan con la consulta dada.



```
db.new_movies.deleteMany({"title" : {"$regex": "^movie"}})
```

Pasar un documento de consulta vacío equivale a no pasar ningún filtro y, por lo tanto, todos los documentos coinciden. La función `deleteOne` eliminará el documento que se encuentre primero. La función `deleteMany` eliminará todos los documentos de la colección.

```
db.new_movies.deleteOne ({})  
db.new_movies.deleteMany ({})
```

Siempre debe asegurarse de que no haya errores tipográficos en el nombre del campo. Un nombre de campo incorrecto puede dar lugar a la eliminación de todos los documentos de la colección.

Busca y elimina

Con **`findOneAndDelete()`** busca y elimina un documento de la colección. Si se encuentra más de un documento, solo se eliminará el primero. Una vez eliminado, **muestra el documento eliminado** como respuesta. En el caso de coincidencias de varios documentos, la opción de **sort** se puede utilizar para influir en qué documento se elimina. La proyección se puede utilizar para incluir o excluir campos del documento en respuesta.

3.4.- Reemplazar y actualizar documentos

El campo **`_id`** sirve como identificador único de un documento y, por lo tanto, no debe cambiarse mientras exista el documento.

MongoDB proporciona el método **`replaceOne()`**, que acepta un filtro de consulta y un documento de reemplazo. El primer argumento es el filtro de consulta para identificar el documento que se reemplazará y el segundo argumento es el nuevo documento.

```
db.users.insertMany([  
  {"_id": 2, "name": "Jon Snow", "email": "Jon.Snow@got.es"},  
  {"_id": 3, "name": "Joffrey Baratheon", "email": "Joffrey.Baratheon@got.es"},  
  {"_id": 5, "name": "Margaery Tyrell", "email": "Margaery.Tyrell@got.es"},  
  {"_id": 6, "name": "Khal Drogo", "email": "Khal.Drogo@got.es"}  
])
```

```
db.users.replaceOne(  
  {"_id" : 5},  
  {"name": "Margaery Baratheon", "email": "Margaery.Baratheon@got.es"}  
)
```



Para modificar uno o solo algunos campos de un documento, MongoDB proporciona el comando de **updateOne()**. Devuelve las estadísticas de la consulta, como cuántos registros coincidieron y cuántos registros se modificaron.

```
db.movies.updateOne(
  {"title" : "Macbeth"},
  {$set : {"year" : 2015}}
)
```

Si al realizar la actualización, incluimos un campo que no existe en el documento, este se añade al documento.

```
db.movies.updateOne(
  {"title" : "Macbeth"},
  {$set : {"type" : "movie", "num_mflix_comments" : 1}}
)
```

Operadores de actualización

- **\$set** se usa para establecer los valores de los campos en un documento o agregar nuevos campos.
- **\$inc** se utiliza para **incrementar** o **decrementar** el valor de un campo numérico en un número específico.
- **\$mul** se utiliza para multiplicar el valor de un campo numérico en un número específico.
- **\$rename** se usa para cambiar el nombre de los campos. Si el campo aún no está presente en el documento, el operador lo ignora y no hace nada. El campo proporcionado y su nuevo nombre deben ser diferentes. Si son iguales, la operación falla con un error. Si un documento ya contiene un campo con el nuevo nombre proporcionado, se eliminará el campo existente.

```
db.movies.findOneAndUpdate(
  {"title" : "Macbeth"},
  {$rename : {"num_mflix_comments" : "comments", "imdb_rating" : "rating"}},
  {returnNewDocument : true}
)
```

- **\$currentDate** se usa para establecer el valor de un campo dado como Date o timestamp. Proporcionar un nombre de campo con un valor de true insertará la fecha actual como una Date. Se puede usar un operador **\$type** para especificar el valor como una Date o timestamp. La notación de puntos anida documentos.

```
db.movies.findOneAndUpdate(  
  {"title" : "Macbeth"},  
  {$currentDate : {  
    "created_date" : true,  
    "last_updated.date" : {$type : "date"},  
    "last_updated.timestamp" : {$type : "timestamp"},  
  }},  
  {returnNewDocument : true}  
)
```

- **\$unset** elimina todos los campos dados del documento coincidente. A medida que se eliminan los campos proporcionados, sus valores especificados no tienen ningún impacto.

3.5.- Operaciones con arrays

- **\$push** añade un elemento a un array existente.

```
db.movies.findOneAndUpdate (  
  {_id: 111},  
  {$push: {"genre": "Drama"}},  
  {"returnNewDocument": true}  
)
```

- **\$push** y **\$each** se usan conjuntamente para añadir más de 1 elemento a la vez a un array

```
db.movies.findOneAndUpdate(  
  {_id : 111},  
  {$push : {  
    "genre" : {  
      $each : ["History", "Action"]  
    }}  
  },  
  {"returnNewDocument" : true}  
)
```

- **\$sort** ordena los elementos de un campo de tipo array del documento.

```
db.movies.findOneAndUpdate(
  { _id : 111 },
  { $push : {
    "genres" : {
      $each : [],
      $sort : 1
    }
  } },
  { "returnNewDocument" : true }
)
```

- **\$addToSet** es como \$push, con la única diferencia de que un elemento se guardará solo si aún no está en el array. Se puede combinar con \$each

```
db.movies.findOneAndUpdate(
  { _id : 111 },
  { $addToSet : { "genre" : "Action" } },
  { "returnNewDocument" : true }
)
```

- **\$pop**, cuando se usa en un comando de actualización, le permite eliminar el primer o último elemento de un array. Elimina un elemento a la vez y solo se puede usar con los valores 1 (para el último elemento) o -1 (para el primer elemento):

```
db.movies.findOneAndUpdate(
  { _id : 111 },
  { $pop : { "genre" : 1 } },
  { "returnNewDocument" : true }
)
```

- **\$pullAll** elimina varios elementos (array) a la vez.

```
db.movies.findOneAndUpdate(
  { _id : 111 },
  { $pullAll : { "genre" : ["Action", "Crime"] } },
  { "returnNewDocument" : true }
)
```

- **.\$** se puede usar para acceder a posiciones del array y realizar todas las operaciones anteriores.

4.- Data Aggregation

Las agregaciones en MongoDB son similares a la **subconsultas** de MySQL. El elemento clave de la agregación se llama **pipeline**. Un **pipeline** es una serie de instrucciones, donde la entrada a cada instrucción es la salida de la anterior.

La canalización se puede pasar de dos formas, ya sea como una variable guardada o directamente como un comando.

```
var pipeline = [  
  { $match: { "location.address.state": "MN" } },  
  { $project: { "location.address.city": 1 } },  
  { $sort: { "location.address.city": 1 } },  
  { $limit: 3 }  
];  
  
db.theaters.aggregate(pipeline)
```

Pasándolo directamente al comando aggregate, la salida se verá de la siguiente manera:

```
db.theaters.aggregate([  
  { $match: { "location.address.state": "MN" } },  
  { $project: { "location.address.city": 1 } },  
  { $sort: { "location.address.city": 1 } },  
  { $limit: 3 }  
]);
```

Podemos crear funciones con agregaciones

```
var simpleFindAsAggregate = function() {  
  print ("Aggregation Result:")  
  var pipeline = [  
    { $match: { "location.address.state": "MN" } },  
    { $project: { "location.address.city": 1 } },  
    { $sort: { "location.address.city": 1 } },  
    { $limit: 3 }  
  ];  
  db.theaters.aggregate(pipeline).forEach(printjson);  
};  
  
simpleFindAsAggregate();
```




Agrupaciones

El comando **\$group** le permite agrupar (o agregar) documentos en función de una condición específica.

La implementación básica del comando **\$group** acepta solo una clave **_id**, siendo el valor una expresión. Esta expresión define los criterios por los cuales la canalización agrupa los documentos. Este valor se convierte en el **_id** del documento recién generado con un documento generado para cada **_id** único que crea el comando **\$group**.

Al agregar, necesitamos decirle a la canalización que queremos acceder al campo del documento que está agregando actualmente.

En el siguiente código agrupará todas las películas por su clasificación, generando un solo registro para cada categoría de clasificación:

```
var pipeline = [
  { $group: {
    _id: "$rated"
  }}
];

db.movies.aggregate(pipeline).forEach(printjson);
```

Acumuladores

El comando **\$group** puede aceptar más de un argumento. También puede aceptar cualquier número de argumentos adicionales en el siguiente formato:

field: { \$accumulator: expression }

- **field** definirá la clave de nuestro campo recién calculado para cada grupo.
- **accumulator** debe ser un operador de acumulador compatible. Se trata de un grupo de operadores, como otros operadores con los que ya puede haber trabajado, como **\$lte**, excepto que, como sugiere el nombre, acumularán su valor en varios documentos que pertenecen al mismo grupo.
- **expression** en este contexto se pasará al operador del acumulador como la entrada de qué campo en cada documento debería estar acumulando.

```
var pipeline = [
  {$group: {
    _id: "$rated",
    "numTitles": {$sum: 1},
  }}
];

db.movies.aggregate(pipeline).forEach(printjson);
```

Del mismo modo, en lugar de acumular 1 en cada documento, puede acumular el valor de un campo determinado.

```
var pipeline = [
  {$group: {
    _id: "$rated",
    "sumRuntime": { $sum: "$runtime"},
  }}
];

db.movies.aggregate(pipeline).forEach(printjson);
```

Podemos cambiar nuestro acumulador de \$sum a \$avg, que devolverá el tiempo de ejecución promedio en cada grupo, por lo que nuestra canalización se convierte en el siguiente:

```
var pipeline = [
  {$group: {
    _id: "$rated",
    "avgRuntime": { $avg: "$runtime"},
  }}
];

db.movies.aggregate(pipeline).forEach(printjson);
```

5.- Joins \$lookup

En MongoDB, los joins de colecciones se realizan mediante el paso de agregación **\$lookup**.

- **from:** La colección que estamos joining a nuestra agregación actual. En este caso, estamos trayendo los comentarios a los usuarios.



- **localField**: el nombre del campo que vamos a utilizar para unir nuestros documentos en la colección local (la colección en la que estamos ejecutando la agregación: users). En este caso, el **name** de nuestro usuario.
- **foreignField**: el campo que enlaza con localField en la colección from (comments). Estos pueden tener diferentes nombres, pero en este escenario, es el mismo campo: name.
- **as**: así es como se etiquetarán nuestros nuevos datos combinados.

```
var lookupExample = function() {
  var pipeline = [
    { $match: { $or: [{"name": "Catelyn Stark"}, {"name": "Ned Stark"}]}},
    { $lookup: {
      from: "comments",
      localField: "name",
      foreignField: "name",
      as: "comments"
    }},
  ],
  { $limit: 2},
];
db.users.aggregate(pipeline).forEach(printjson);
}

lookupExample();
```

Referencia web:

https://github.com/zpio/NoSql_con_MongoDB